

git

Prácticas — parte 1: trabajando en local

Antes de empezar

- Abre una terminal.
- Todos los ejercicios se hacen **en local**, no hace falta ni cuenta de GitHub ni conexión a internet.
- Ve resolviendo los ejercicios en orden, cada uno se apoya en el anterior.
- Los **retos** (al final) no traen los pasos, solo el objetivo: decides tú los comandos.

Ejercicios guiados

Ejercicio 1 — Identifícate

1. Comprueba si ya tienes configurado tu nombre y tu email en git.
2. Si no lo están (o quieres cambiarlos), configúralos de forma **global**.
3. Vuelve a comprobar que se ha guardado correctamente.

Resultado esperado: `git config --global --list` muestra tu `user.name` y tu `user.email`.

Ejercicio 2 — Tu primer repositorio

1. Crea una carpeta llamada `practica-git`.
2. Entra en ella.
3. Conviértela en un repositorio de git.
4. Comprueba que se ha creado correctamente sin usar `ls -a` (piensa qué comando de los vistos en clase te lo dice).

Resultado esperado: `git status` muestra información. Un `git status` en una carpeta sin `git init` no funciona.

Ejercicio 3 — Ignora lo que no toca

1. Dentro de `practica-git`, crea un fichero `notas.txt` con el texto que quieras.
2. Crea también un fichero `debug.log` (puede estar vacío).
3. Crea un `.gitignore` que ignore todos los ficheros `.log`.
4. Comprueba con `git status` que `debug.log` **no** aparece como untracked, pero `notas.txt` sí.

Resultado esperado: `git status` solo menciona `notas.txt` y `.gitignore`.

Ejercicio 4 — Tu primer commit

1. Añade `notas.txt` al área de preparación.
2. Comprueba con `git status` que ha cambiado de sección (untracked → staged).
3. Confirma el cambio con un mensaje descriptivo.
4. Comprueba con `git log` que el commit aparece.

Resultado esperado: `git log` muestra 1 commit con tu nombre y tu mensaje.

Ejercicio 5 — El ciclo completo

1. Modifica el contenido de `notas.txt` (añade una línea nueva).
2. Ejecuta `git status`. ¿En qué sección aparece ahora `notas.txt` ?
3. Añade también el `.gitignore` que creaste en el ejercicio 3 (si aún no lo habías confirmado).
4. Prepara **todos** los cambios pendientes de una sola vez (usa el comando que añade todo el directorio).
5. Haz un commit.

Resultado esperado: `git log --oneline` muestra 2 commits.

Ejercicio 6 — Revisa el historial

1. Crea 2 ficheros nuevos más (`archivo1.txt` , `archivo2.txt`) y confírmalos **en un único commit**.
2. Mira el historial completo con `git log` .
3. Mira el historial compacto con `git log --oneline` .
4. Identifica: ¿cuál es el hash corto del primer commit que hiciste (ejercicio 4)?

Resultado esperado: sabes leer y diferenciar la salida de `git log` y `git log --oneline` .

Retos

Aquí no hay pasos: piensa qué comandos necesitas.

Reto 1 — Commit selectivo

Modifica a la vez `notas.txt`, `archivo1.txt` y `archivo2.txt`.

Objetivo: confirma en un commit **solo** los cambios de `notas.txt` y `archivo1.txt`.
`archivo2.txt` debe seguir apareciendo como modificado (sin confirmar) después del commit.

Comprueba el resultado con `git status` y `git log`.

Reto 2 — Todos los estados a la vez

Objetivo: deja tu repositorio en un estado en el que `git status` muestre **a la vez**:

- Un fichero **untracked** (nuevo, nunca añadido).
- Un fichero **staged** (preparado, no confirmado).
- Un fichero **modified** ya rastreado pero con cambios sin preparar.

Haz una captura o copia de la salida de `git status` y explica, para cada fichero, por qué está en ese estado.

Reto 3 — El .gitignore que se resiste

Crea un fichero llamado `config.env` con contenido inventado (simulando una contraseña) y añádelo con `git add`.

Después, añade `config.env` a tu `.gitignore`.

Objetivo: consigue que `config.env` deje de aparecer en `git status` como pendiente de cambios. ¿Por qué el `.gitignore` no bastaba por sí solo? ¿Qué comando adicional has necesitado?

Reto 4 — Antes de confirmar, revisa

Modifica `notas.txt` sin usar aún `git add`.

Objetivo: usa el comando adecuado para ver, línea a línea, exactamente qué has cambiado respecto a la última versión confirmada — sin ejecutar `git add` todavía.

Reto 5 — Reconstruye la historia

Al terminar todos los ejercicios y retos anteriores:

1. Ejecuta `git log --oneline` y copia el resultado.
2. Escribe, commit a commit, una frase que resuma qué se hizo en cada uno (a partir solo del mensaje, sin mirar el código).

Objetivo: comprobar que sabes escribir (y leer) mensajes de commit útiles.

Autoevaluación

¿Te ves capaz de...?

- [] Explicar la diferencia entre *working directory*, *staging area* y *repository*.
- [] Explicar la diferencia entre un fichero *untracked*, *modified* y *staged*.
- [] Crear un repositorio desde cero y hacer tu primer commit sin mirar apuntes.
- [] Escribir un `.gitignore` básico para un proyecto nuevo.
- [] Leer la salida de `git log` y `git log --oneline` sin ayuda.
- [] Explicar por qué `git add .` puede ser peligroso sin un `.gitignore`.